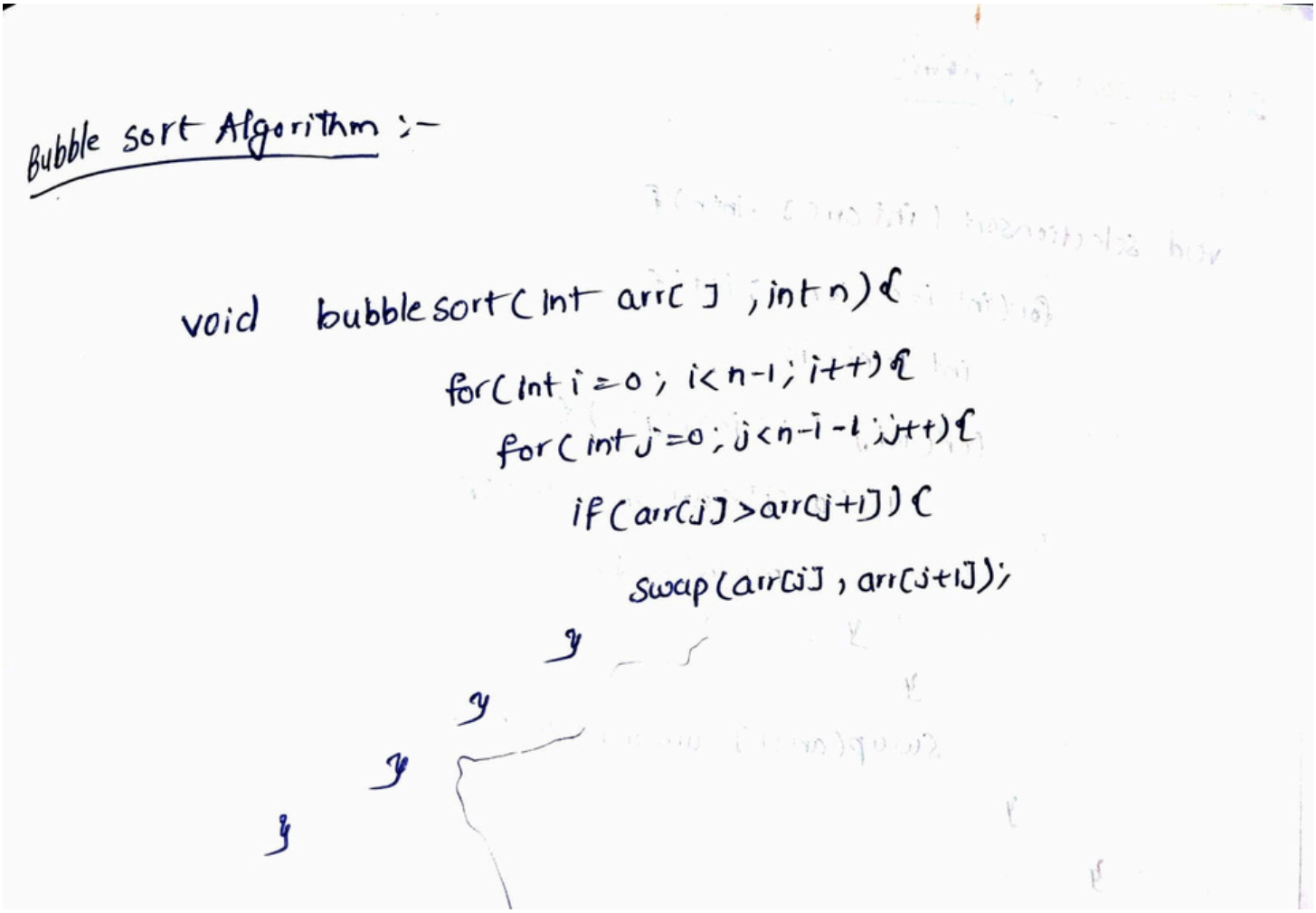
Practical-1

Aim:

To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

1)Bubble sort



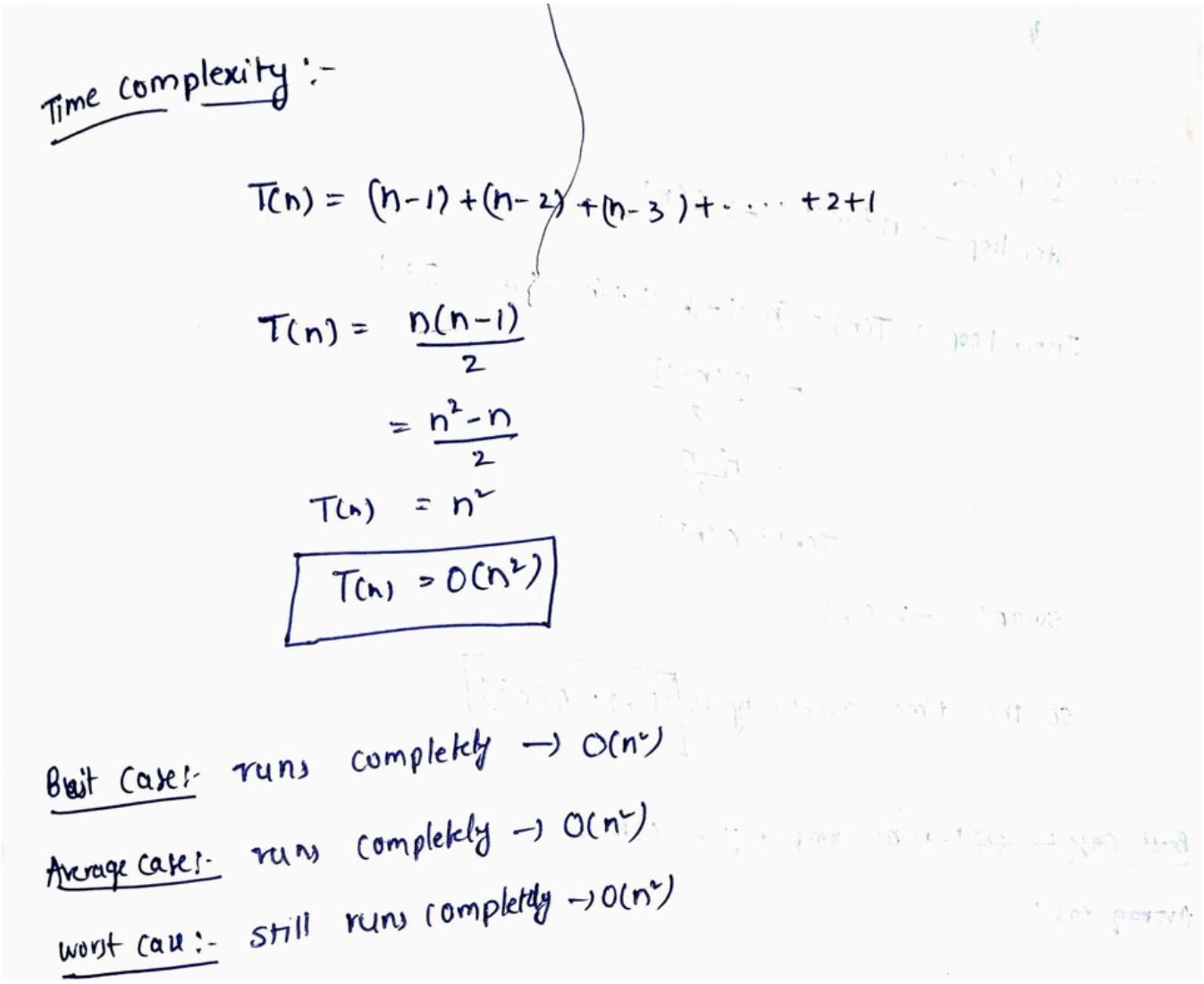
code:

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        // Flag to check if swapping happens
        bool swapped = false;

        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap adjacent elements
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

    // If no swapping occurred, array is already sorted
        if (!swapped)
            break;
    }
}
 int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    bubbleSort(arr, n);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:

```
Output

Sorted array: 11 12 22 25 34 64 90

=== Code Execution Successful ===
```

Time complexity:



2)Selection sort

Selection sort Algorithm:-

```
void selectionsort ( int arr[] , intn) {
        for(int i=0 ; i<n-1 ; i++) {
            int mini = i;
            for(intj=i+1 ; j<n ; j++) {
                if (arr[j] < arr [minIndex]) {
                    minIndex = j;
                }
            }
            swap(arr[i], arr[minIndex]);
        }
}
```

code:

```
#include <iostream>
using namespace std;

void selectionSort(int arr[], int n) {
   for (int i = 0; i < n - 1; i++) {
      int minIndex = i;

      // Find the smallest element
      for (int j = i + 1; j < n; j++) {
         if (arr[j] < arr[minIndex]) {
            minIndex = j;
         }
      }

      // Swap smallest element with current element
      swap(arr[i], arr[minIndex]);
   }
}

int main() {
   int arr[] = {64, 25, 12, 22, 11};
   int n = sizeof(arr) / sizeof(arr[0]);

   selectionSort(arr, n);

   cout << "Sorted array(Selection sort): ";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }

   return 0;
}
```

Output:

```
Output

Sorted array(selection sort): 11 12 22 25 64

=== Code Execution Successful ===
```

Time complexity:

Time complexity:-

Outer loop → n times → O(n)

Inner Loop → $T(n) = (n-1)+(n-2)+(n-3)+\cdots 2+1$

$= \frac{n(n-1)}{2}$

$= \frac{n^2-n}{2}$

$T(n) = O(n^2)$

swap() → O(1)

So the time complexity is $\boxed{T(n) = O(n^2)}$

3)Insertion sort

Insertion Sort Algorithm

```
void insertionSort(int arr[], int n){
    for(int i=1; i<n; i++){
        int key = arr[i];
        int j = i-1;
        while(j>=0 && arr[j] > key){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

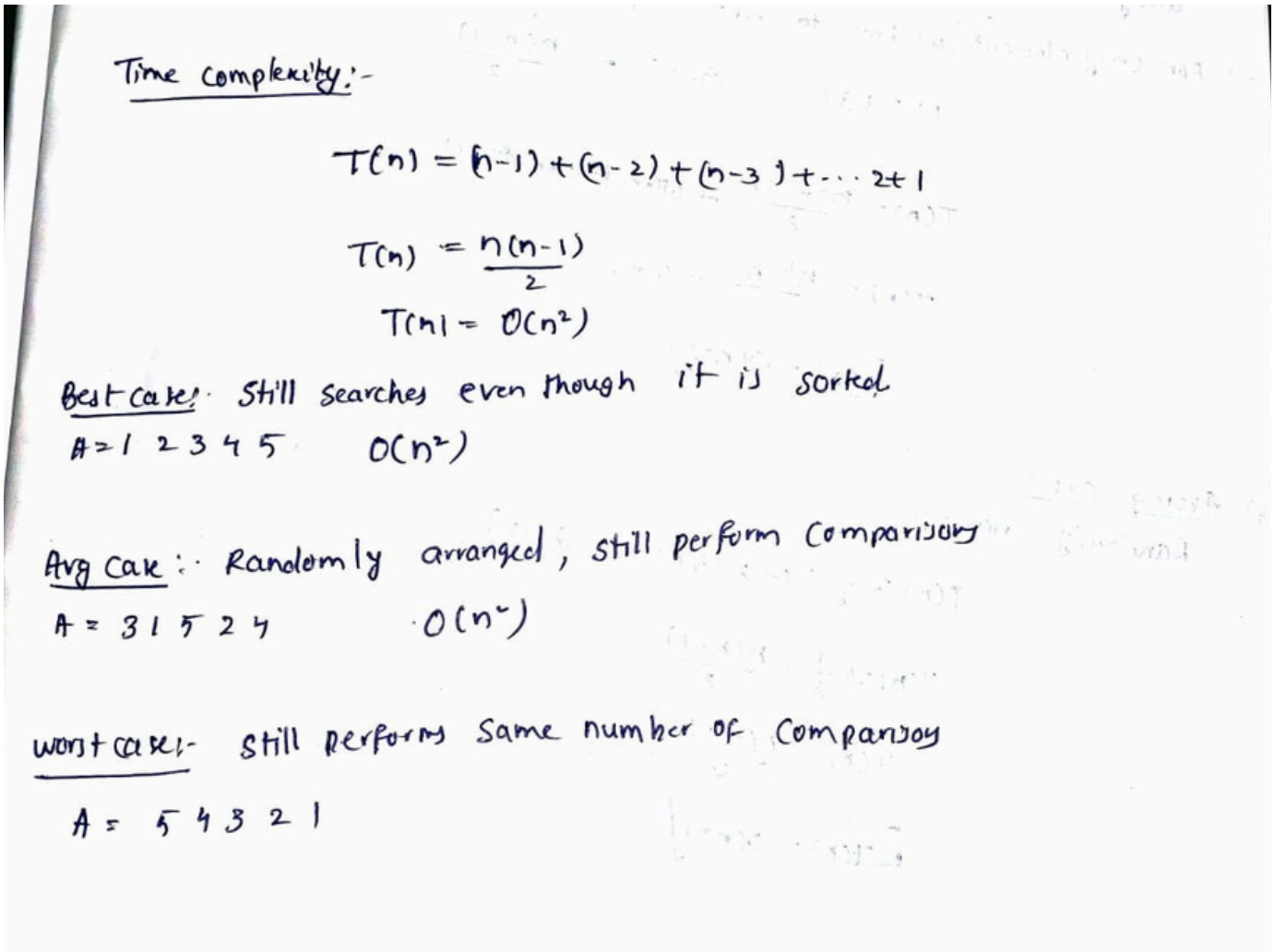
code:

```
#include <iostream>
using namespace std;

void insertionSort(int arr[], int n) {
   for (int i = 1; i < n; i++) {
      int key = arr[i];
      int j = i - 1;

      // Move elements greater than key one position ahead
      while (j >= 0 && arr[j] > key) {
         arr[j + 1] = arr[j];
         j--;
      }

      // Insert key at the correct position
      arr[j + 1] = key;
   }
}

int main() {
   int arr[] = {12, 11, 13, 5, 6};
   int n = sizeof(arr) / sizeof(arr[0]);

   insertionSort(arr, n);

   cout << "Sorted array(insertion sort): ";
   for (int i = 0; i < n; i++) {
      cout << arr[i] << " ";
   }

   return 0;
}
```

Output:

```
Output
Sorted array(insertion sort): 5 6 11 12 13

=== Code Execution Successful ===
```

Time complexity:



4)Merge sort

merge sort:-

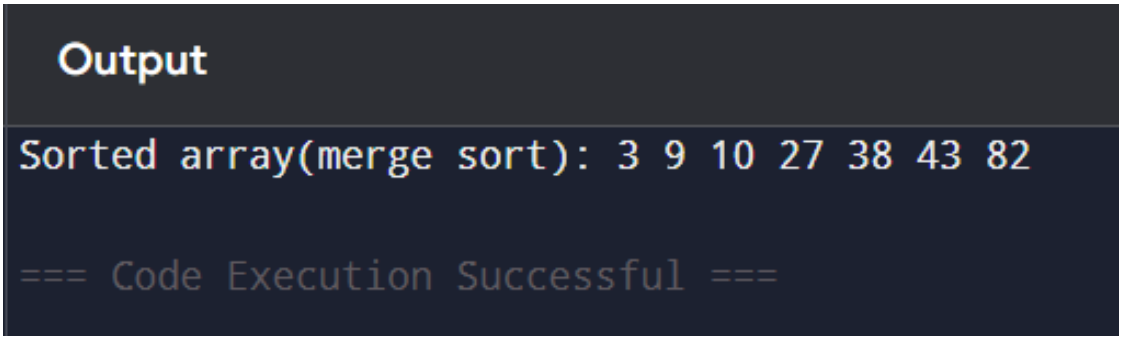
```
void merge(int arr[], int l, int m, int r) {
    int n1 = m-l+1;
    int n2 = r-m;
    vector<int> L(n1), R(n2);
    for (int i=0; i<n1; i++) {
        L[i] = arr[l+i];
    }
    for (int j=0; j<n2; j++) {
        R[j] = arr[m+1+j];
    }
    int i=0, j=0, k=l;
    while (i<n1 && j<n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        }
        else {
            arr[k++] = R[j++];
        }
    }
    while (i<n1) arr[k++] = L[i++];
    while (j<n2) arr[k++] = R[j++];
}
void mergesort(int arr[], int l, int r) {
    if (l<r) {
        int m = l+(r-l)/2;
        mergesort(arr, l, m);
        mergesort(arr, m+1, r);
        merge(arr, l, m, r);
    }
```

code:

```
#include <iostream>
using namespace std;

void merge(int arr[], int low, int mid, int high) {
   int i = low;
   int j = mid + 1;
   int k = 0;

   int temp[100];

   while (i <= mid && j <= high) {
      if (arr[i] <= arr[j]) {
         temp[k] = arr[i];
         i++;
      } else {
         temp[k] = arr[j];
         j++;
      }
      k++;
   }

   while (i <= mid) {
      temp[k] = arr[i];
      i++;
      k++;
   }

   while (j <= high) {
      temp[k] = arr[j];
      j++;
      k++;
   }

   for (i = low, k = 0; i <= high; i++, k++) {
      arr[i] = temp[k];
   }
}
```

```
    void mergeSort(int arr[], int low, int high) {
        if (low < high) {
            int mid = (low + high) / 2;

            // Divide
            mergeSort(arr, low, mid);
            mergeSort(arr, mid + 1, high);

            // Merge
            merge(arr, low, mid, high);
        }
    }


int main() {
    int arr[] = {38, 27, 43, 3, 9, 82, 10};
    int n = sizeof(arr) / sizeof(arr[0]);

    mergeSort(arr, 0, n - 1);

    cout << "Sorted array(merge sort): ";

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:



Time complexity:

Merge sort Time Complexity :-

$$T(n) = 2T(\tfrac{n}{2}) + n$$

$$T(n) = 2[2T(n/4) + n/2] + n$$

$$T(n) = 4T(n/4) + 2n$$

$$T(n) = 8T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$\frac{n}{2^k} = 1$$

$$2^k = n$$

$$k = \log_2 n$$

So.

$$T(n) = nT(1) + n\log_2 n$$

$$T(n) = n + n\log n$$

$$\boxed{T(n) = O(n\log n)}$$

Best case- $O(n\log n)$

Avg case:- $O(n\log n)$

worst case:- $O(n\log n)$

-) It remains same for all case it still divides in recursio

5)Quick sort

Quick Sort Algorithms

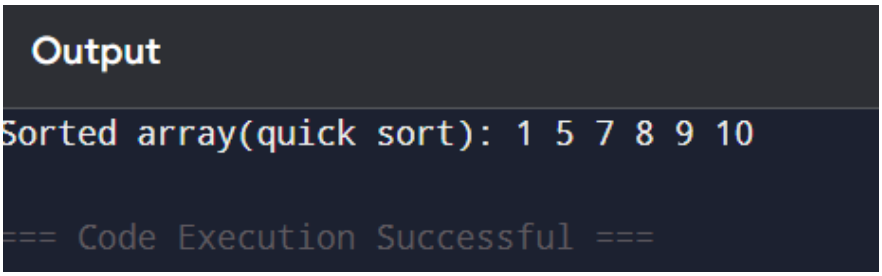
```
int partition(int arr[], int low, int high) {
        int pivot = arr[low];
        int i = low;
        int j = high;
    while (i<j) {
        while (arr[i]<= pivot && i<= high-1) { i++; };
        while ( arr[j] > pivot && j>= low+) { j--; };
      if(i<j) {
          swap( arr[i], arr[j]);
       }
    }
     swap(arr[low], arr[j]);

  return j;

}

void quicksort (int arr[], int low, int high) {
      if ( low < high) {
        int partition_index = partition (arr, low, high);
        quicksort (arr, low, partition_index - 1);
        quicksort (arr, partition_index+1, high);
      }
}
```

code:

```
#include <iostream>
using namespace std;

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}



void quickSort(int arr[], int low, int high) {
                    if (low < high) {
        int pi = partition(arr, low, high);

                        // Divide
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
                        }
                    }
```

```
int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    quickSort(arr, 0, n - 1);

    cout << "Sorted array(Quick sort): ";

    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }

    return 0;
}
```

Output:



Time complexity:

Time Complexity:-

1) Best case:-

$$T(n) = n \log_2 n$$

$$\boxed{T(n) = O(n \log n)}$$

2) Avg case:-

$$T(n) = 2T(n/2) + n$$

$$T(n) = 2[2T(n/4) + n/2] + n$$

$$T(n) = 4T(n/4) + 2n$$

$$T(n) = 8T(n/8) + 3n$$

$$T(n) = 2^k T(n/2^k) + kn$$

$$\frac{n}{2^k} = 1$$

$$k = \log_2 n$$

$$T(n) = n \log_2 n$$

$$\boxed{T(n) = O(n \log n)}$$

3) Worst case:- Instead of dividing into two equal parts, we get

$$T(n) = n + (n-1) + (n-2) + \cdots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

$$= \frac{n^2 + n}{2}$$

$$\boxed{T(n) = O(n^2)}$$

Conclusion:

We implemented Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, and Quick Sort and studied their working and time complexities. Merge Sort and Quick Sort are generally more efficient for large datasets compared to the other three algorithms.